Assignment - 1.3

P.HARSHAVARDHAN
2303A53038
BATCH – 46

Task 1: AI-Generated Logic Without Modularization (Fibonacci Sequence Without Functions)

INPUT:

```python
# write a python program to print fibonacci series up to n terms without using functions
n = int(input("Enter the number of terms: "))
a, b = 0, 1
if n <= 0:
    print("Please enter a positive number")
elif n == 1:
    print(a)
else:
    print(a, end=" ")
    print(b, end=" ")
    for i in range(2, n):
        c = a + b
        print(c, end=" ")
        a = b
        b = c
    print()
```

OUTPUT:

```
PS C:\Users\anagandula gouthami\OneDrive\Desktop\AI Coding> &
/Users/anagandula gouthami/OneDrive/Desktop/AI Coding/2303A53
Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Program to print Fibonacci series without using any function

User enters number of terms

Fibonacci logic is written directly in main code

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

INPUT:

```python
# Optimize and simplify logic Fibonacci code series up to n terms without using functions
n = int(input("Enter the number of terms: "))
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()
```

OUTPUT:

```
/Users/anagandula goutham1/OneDrive/Desktop/AI
Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
Enter the number of terms:
```

COMMENTS:

Optimized Fibonacci program

Redundant variables are removed

Tuple assignment is used for better readability

Code is simple and efficient

Task 3: Modular Design Using AI Assistance (Fibonacci Using Functions)

INPUT:

```python
# display a function to generate Fibonacci series up to n terms in Python
def fibonacci_series(n):
    a, b = 0, 1
    series = []
    for _ in range(n):
        series.append(a)
        a, b = b, a + b
    return series
n = int(input("Enter the number of terms: "))
print(fibonacci_series(n))
```

OUTPUT:

```
Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

COMMENTS:

Program to generate Fibonacci series using a function

Function improves reusability and modularity

Same logic can be reused in multiple programs

Task 4: Comparative Analysis – Procedural vs Modular Fibonacci Code

INPUT:

```
# display and compare Fibonacci program without function and with function
n = int(input("Enter the number of terms: "))
# Without function
a, b = 0, 1
print("Fibonacci series without function:")
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()
# With function
def fibonacci_series(n):
    a, b = 0, 1
    series = []
    for _ in range(n):
        series.append(a)
        a, b = b, a + b
    return series
print("Fibonacci series with function:")
print(*fibonacci_series(n))
```

OUTPUT:

```
Enter the number of terms: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
Enter the number of terms: 6
0 1 1 2 3 5
```

COMMENTS:

Comparison between Fibonacci program with function and without function

Function-based code is more reusable and readable

Procedural code is simple but not suitable for large programs

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series)

INPUT:

```
# display and write fibonacci program using loop
n = int(input("Enter the number of terms: "))
a, b = 0, 1
print("Fibonacci series using loop:")
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()
```

OUTPUT:

```
Enter the number of terms: 7
0 1 1 2 3 5 8
```

COMMENTS:

Fibonacci series using iterative approach

Uses loop to generate Fibonacci numbers

Faster and memory efficient